

PENCH LOGISTICS ERP

System Architecture & Data Dependency Manual

OPERATIONAL DEPENDENCY & DATA FLOW

A technical guide mapping database schemas, model hierarchies, and runtime dependencies required to perform dairy logistics dispatches, route planning, geofenced zones creation, and daily reconciliation.

Prepared By: Antigravity AI Engineering

Target Workspace: Pench Foods Backend Core

Published: May 2026

1. Operational Life Cycle & Flow Tree

The diagram below outlines the logical sequence of operations, moving from base city creation to actual last-mile driver trip dispatching and reconciliation. An object cannot be instantiated unless all connected parent nodes are present in the database.

```
[ PUBLIC SCHEMA (Global Setup) ]
|
+---> tenants.City (e.g. Pune, Nagpur)
      |
      +---> tenants.Domain (e.g. pune.penchfoods.com)
            |
            v
[ TENANT SCHEMA (Specific City Area) ]
|
+---> routing.Zone (Defines regional boundaries)
      |
      +---> crm.Customer (Address, geocoded points, zone association)
            |
            +---> subscriptions.Subscription (Active repeat schedule)
                  |
                  +---> orders.Order (Generated delivery jobs at 4 AM)
+---> inventory.Product (Warehouse items catalogue)
|
+---> routing.Driver (Links User account to Vehicle and operational Zone)
      |
      v
[ DAILY OPERATIONS WORKFLOW (Dispatches) ]
|
+---> routing.Route (Assigned Driver + target Orders; generated at 6 AM)
      |
      +---> tracking.DriverTrail (WebSocket snap-to-road trails)
      +---> routing.TrackingEvent (Geotagged POD signatures & photos)
      +---> routing.DailyReconciliation (Final cash/UPI balances verified)
```

Core Rule & Constraint Example:

Without a registered 'Driver' profile and at least one pending 'Order' assigned inside the driver's geofenced 'Zone', a logistics 'Route' cannot be compiled. The OSM/OR-Tools optimization engine requires coordinates of active orders and driver vehicle capabilities before drawing routing lines on the dispatcher map.

2. Step-by-Step Data Onboarding Guide

Follow this exact deployment and data entry workflow when launching a new city branch. Skipping steps will cause database validations to fail or prevent the automated scheduling engine from processing orders.

Step 1: Base Tenant Infrastructure

- * **City (tenants.City):** Stores the tenant schema ID. Creating this triggers django-tenants to spin up an isolated schema with empty tables.
- * **Domain (tenants.Domain):** Maps a domain/subdomain to the city. Needed so the public routing engine knows which Postgres schema to query.

Step 2: Regional Foundations

- * **Zone (routing.Zone):** A polygon mapping delivery sectors. Helps isolate customers and pre-assign permanent local drivers.
- * **Product (inventory.Product):** The catalog items. Can optionally specify a returnable BottleType with deposit values for dairy container tracking.

Step 3: Core Profiles onboarding

- * **User Accounts (accounts.User):** Global login credential (phone, role, password). Must be created before linking to a specific operational role profile.
- * **Driver Profiles (routing.Driver):** Binds the global user to a specific vehicle plate, weight capacity, and geographic Zone.
- * **Customer Profiles (crm.Customer):** Houses delivery instructions, exact GIS coordinates, QR codes, and connects the user account to a Zone.

Step 4: Subscriptions & Run Workflow

- * **Subscription (subscriptions.Subscription):** Defines frequency (Daily, Alternate days, custom JSON list). Generates pending Orders every day at 4 AM.
- * **Route Planning (routing.Route):** Created at 6 AM. Collects all pending Orders, calculates the optimized path, and links to the Driver.

3. System Model Relationships Reference

Below is the technical matrix of model dependencies inside the Pench Logistics system. Ensure that before calling POST endpoints for any resource listed below, all dependent objects have been successfully seeded.

Model	Primary Dependencies	Relation Type	Consequence If Missing / Purpose
City	None	Base	No tenant PostgreSQL schemas can exist.
Domain	City	STRICT	Subdomains route to 404; clients cannot reach API.
Zone	City	Optional	Cannot geofence boundaries or auto-dispatch drivers.
Driver	User, Zone	STRICT	Cannot assign vehicle plate or optimize logistics capacity.
Customer	User, Zone	STRICT	Order coordinates fail; system cannot map address stops.
Subscription	Customer, Product	STRICT	No orders can be auto-generated at 4 AM.
Order	Customer, Product	STRICT	Delivery schedules fail; no POD triggers can be initiated.
Route	Driver, Orders	STRICT	OR-Tools optimization fails; driver has no active trip.
TrackingEvent	Route, Order	STRICT	Cannot record proof photo, signature or POD metadata.
Reconciliation	Driver, Route	STRICT	Cannot balance cash & UPI collections at end-of-shift.
DriverLocation	Driver	STRICT	WebSocket live coordinates and snap trail loops fail.

Concrete Workflow Execution Example:

1. Setting up Pune branch: Create a City 'Pune' (PUN). Add Domain 'pune.penchfoods.com'.
2. Add regional items: Add Product 'Full Cream Milk 500ml' and define delivery 'Zone: Pune-East'.
3. Driver registration: Create User 'driver_pune_1', link to Driver Profile (Plate: MH-12-AB-5678, Zone: Pune-East).
4. Customer registration: Create User 'customer_pune_101', link to Customer Profile (Zone: Pune-East, geolocation coordinates: [18.5204, 73.8567]).
5. Start recurring plan: Add Subscription for 'customer_pune_101' for 2x milk bottles daily. At 4 AM, Celery spawns pending Order #1042.
6. Dispatch Optimization: Admin starts route generation at 6 AM. The system pairs 'driver_pune_1' with 'Order #1042' in 'Zone: Pune-East'. The optimized Route stops sequence is created.
7. Last-mile Execution: Driver hits 'Start Trip' in app, streams live geolocations via WebSocket, scans customer QR code, uploads delivery photo, and hits 'Complete Trip'.